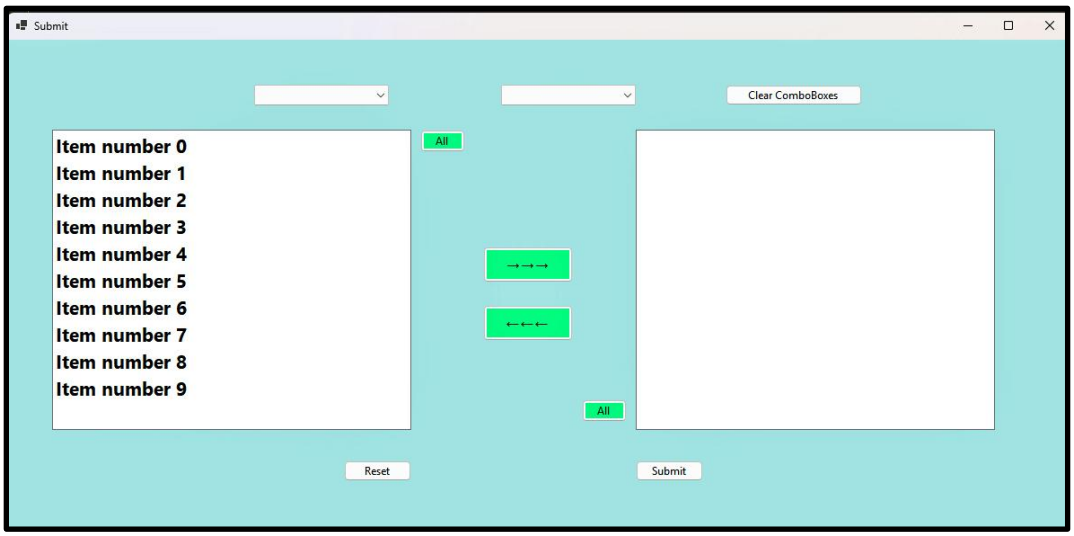
Chapter 13: Multi-Select Wizard and Cascading ComboBoxes

In this lesson we are going to cover the ListBox and ComboBox classes. ListBoxes contain a collection of items, and the user can select zero, one, or many of those items. A ComboBox also contains a collection of items, but the user may only select zero or one of those items.

You have probably encountered a pair of ListBoxes similar to those shown here. There is a number of items in the left-hand box, and you select as many of those items as you like and move them over to the right-hand box for further processing. This configuration of paired ListBoxes is commonly known as a multi-select wizard.

Similarly, at the top of this Form you see a pair of ComboBoxes. The user selects an item from the left-hand ComboBox and their selection determines which items are displayed in the right-hand ComboBox. This configuration is known as cascading dropdowns because the ComboBox class is commonly known as a dropdown box by users.



We will begin with the multi-select wizard. The way the multi-select wizard works is there are two ListBoxes, originally all the items are in the left ListBox. The user selects one, many, or all of the items and moves them to the right ListBox to indicate that those items have been selected for some type of subsequent processing. Once moved to the right ListBox, the user is free to move one, some, or all of the items back to the left, unselected, ListBox. This item exchange continues until the user clicks the reset Button which returns all the items to the left-hand ListBox or the submit Button which moves the right-hand ListBox's items to a subsequent page for some type of further processing.

Let's get started. We begin of course by creating a new Windows Forms application named Chapter 13 Multi-Select Wizard. Please set the properties for your Form as follows:

Form1		
	Name	Wizard
	Size	1200,600
	BackColor	PaleTurquoise (Web tab)
	Text	Multi-Select Wizard

Given that the first half of this example is all about the ListBox class, it will come as no surprise to you that we will begin by dragging one onto our Form from the ToolBox pane. Please set that ListBox's properties as follows:

ListBox1		
	Name	LeftListBox
	Font	Segoe UI, Bold, 16pt
	Location	50,100
	SelectionMode	MultiExtended
	Size	400,350
	Sorted	True

The SelectionMode property of a ListBox may assume any of four values: None, One, MultiSimple or MultiExtended. The None and One options are self-explanatory. MultiSimple requires the user click on each entry they wish to select. MultiExtended allows the user to employ standard Windows multi-selection functionality, i.e. the arrows, CTRL + Click, Shift + Click, CTRL + A.

Once you have your LeftListBox properly configured go ahead and copy and paste that guy to create an exact replica. Set that guy's properties as follows:

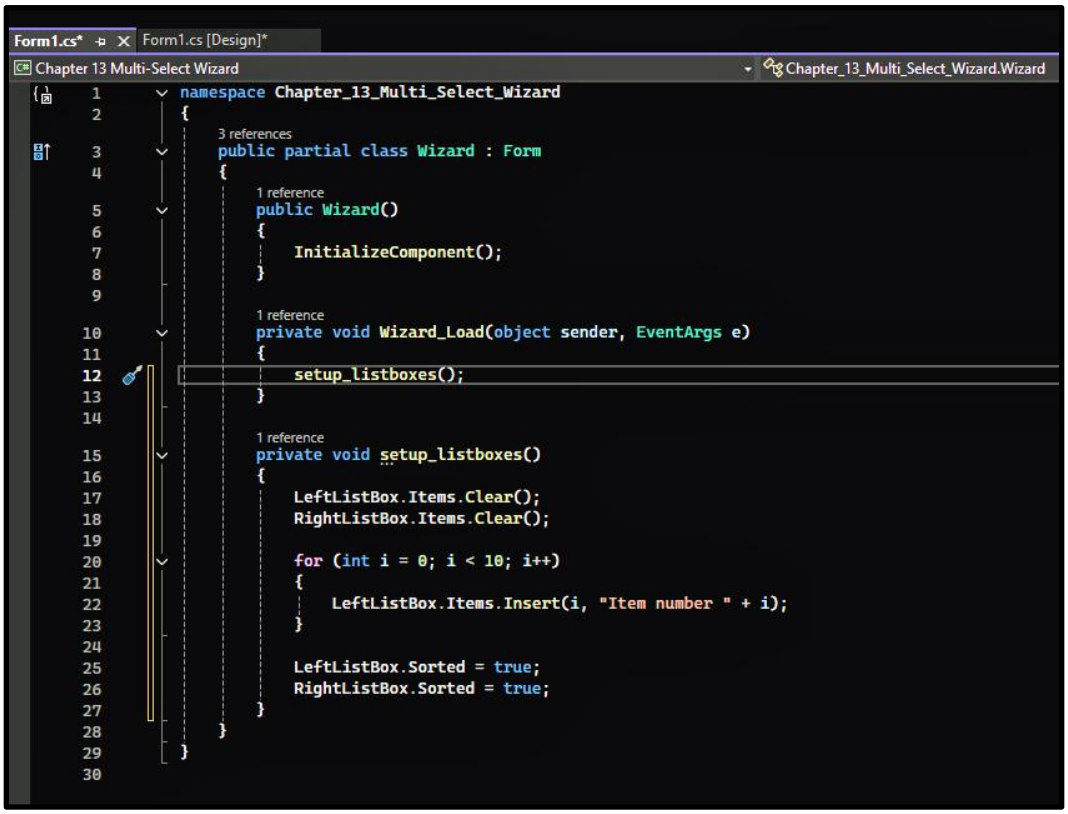
ListBox2		
	Name	RightListBox
	Location	700,100

Now let's add some items to our LeftListBox. We will do so on the load of the form, but since the reset Button will eventually perform the exact same function, we will segregate this code out into a separate routine aptly named setup_listboxes. Your code should look something like this. (Figure 13.1)

The important thing to notice about this code is that the ListBox class displays an indexed collection of items. Given this, it should not be surprizing that we populate a ListBox with a for loop where the loop control variable also serves as the Items collection index.

Please note that under normal conditions, a ListBox would be populated from a database where we read all the rows of a table which meet some condition. We'll take a look at that situation later in this course.

Figure 13.1: setup_listboxes



Next please add a Button to your Form. Set the properties of that Button as follows:

Button1		
	Name	AllLeftButton
	Location	460,100
	Size	50,25
	Text	All
	BackColor	SpringGreen

Copy and paste that button to create a second Button with the following properties:

Button1		
	Name	AllRightButton
	Location	640,400

Copy and paste one of those Buttons twice more to create the following Buttons:

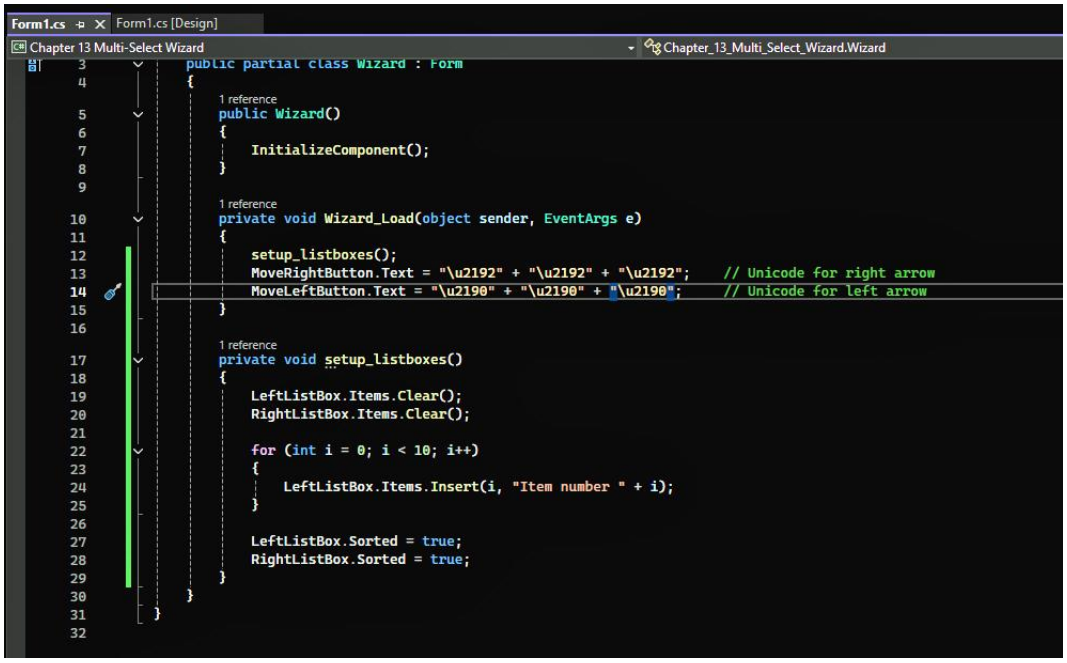
Button1		
	Name	MoveRightButton
	Location	530,230
	Font	Bold, 14 pt
	Size	100,40
	Text	Blank, we will set this on Form Load

Button1		
	Name	MoveLeftButton
	Location	530,290
	Font	Segoe UI, Bold, 14 pt
	Size	100,40
	Text	Blank, we will set this on Form Load

Okay, time to write some code. We begin in the Form Load event handler. (Figure 13.2) Wow, what is that stuff being assigned to the Text property of our MoveLeftButton and MoveRightButtons? Looks like some type of encoded language.

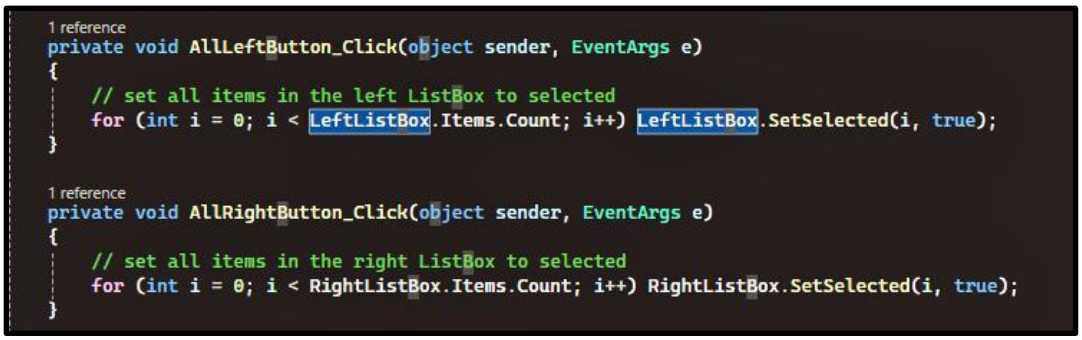
Well, that stuff is Unicode, an extension of the ASCII character set we learned about in an earlier lesson. As the comment suggests, we are using Unicode here to represent the right and left arrows. So how do you know which Unicode to use? Same way you determine all meaningless details in life, Google of course.

Figure 13.2: Unicode



Our code behind the all Buttons is pretty straightforward. (Figure 13.3) If the user clicks one of these Buttons all the items in the associated ListBox are selected. As shown in the example, the syntax for setting an item to selected is ListBoxName.SetSelected(i, true) where i is the index of the item to be selected. To unselect an item the second parameter is set to false.

Figure 13.3: SetSelected



The move routines are a little more interesting. (Figure 13.4) For example, in the MoveRightButton_Click routine we again start out with a for loop walking through each item in the ListBox collection. For each item we first check whether that item is selected by calling the GetSelected method on that index. If the GetSelected method resolves to true we then add that item to the RightListBox's Items collection. The second for loop walks the index tree backwards removing all selected items.

Figure 13.4: Move Routines

```
1 reference
private void AllLeftButton_Click(object sender, EventArgs e)
{
    // set all items in the left ListBox to selected
    for (int i = 0; i < LeftListBox.Items.Count; i++) LeftListBox.SetSelected(i, true);
}

1 reference
private void AllRightButton_Click(object sender, EventArgs e)
{
    // set all items in the right ListBox to selected
    for (int i = 0; i < RightListBox.Items.Count; i++) RightListBox.SetSelected(i, true);
}

1 reference
private void MoveRightButton_Click(object sender, EventArgs e)
{
    // if left ListBox item is selected move it to right ListBox
    for (int i = 0; i < LeftListBox.Items.Count; i++)
    {
        if (LeftListBox.GetSelected(i) == true) // it is selected
            RightListBox.Items.Add(LeftListBox.Items[i]);
    }

    // now reverse the loop for the removes
    for (int i = LeftListBox.Items.Count-1; i >= 0; i--)
    {
        if (LeftListBox.GetSelected(i) == true) LeftListBox.Items.RemoveAt(i);
    }
}

1 reference
private void MoveLeftButton_Click(object sender, EventArgs e)
{
    for (int i = 0; i < RightListBox.Items.Count; i++)
    {
        if (RightListBox.GetSelected(i) == true)
            LeftListBox.Items.Add(RightListBox.Items[i]);
    }

    for (int i = RightListBox.Items.Count-1; i >= 0; i--)
    {
        if (RightListBox.GetSelected(i) == true) RightListBox.Items.RemoveAt(i);
    }
}
```

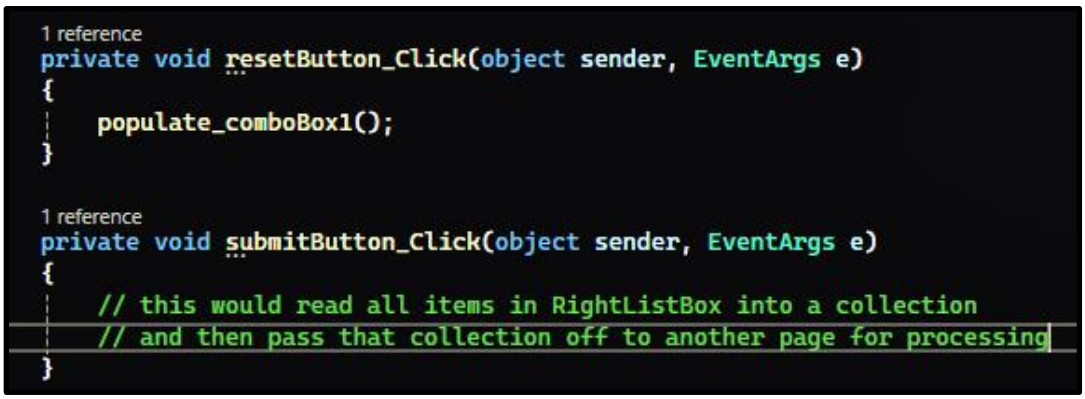
We finish off our multi-select Wizard by adding two Buttons to the bottom of the Form. Their properties are listed below. Please note the guide lines if you set the location of these Buttons using your mouse.

Button1		
	Name	resetButton
	Location	375,468
	Text	Reset

Button1		
	Name	submitButton
	Location	700,468
	Text	Submit

The code behind the reset Button is of course nothing more than our setup_listboxes routine. I have left a stub behind the submit Button click event. A stub is nothing more than a comment saying what code needs to go here in the future. (Figure 13.5)

Figure 13.5: Reset and Submit Buttons



Okay, enough on ListBoxes let's move onto the ComboBox class. ComboBoxes are similar to ListBoxes in that they have an indexed collection of items. They differ from ListBoxes in that the user can only select one item.

We begin by adding two ComboBoxes and a Button to our Form and setting their properties like this:

ComboBox1		
	Name	LeftComboBox
	Location	275,50
	Size	150,23

ComboBox1		
	Name	RightComboBox
	Location	550,50
	Size	150,23

Button1		
	Location	800,50
	Text	Clear ComboBoxes

The code behind these new objects is super straightforward. (Figure 13.6) One thing to note however, despite being an indexed collection of items, the items are not added via that index, yet another Microsoft inconsistency.

Figure 13.6: ComboBoxes

```
private void Wizard_Load(object sender, EventArgs e)
{
    setup_listboxes();
    MoveRightButton.Text = "\u2192" + "\u2192" + "\u2192";    // Unicode for right arrow
    MoveLeftButton.Text = "\u2190" + "\u2190" + "\u2190";     // Unicode for left arrow

    // populate comboBox1 and clear comboBox2
    populate_comboBox1();
    RightComboBox.Items.Clear();
}
1 reference
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    populate_comboBox2();
}

2 references
private void populate_comboBox1()
{
    for (int i = 0; i < 10; i++) LeftComboBox.Items.Add("Number " + i);
}
1 reference
private void populate_comboBox2()
{
    string selectedText = LeftComboBox.SelectedItem.ToString();

    // add the selected item to comboBox2
    RightComboBox.Items.Clear();
    RightComboBox.Items.Add(selectedText);

}

1 reference
```

That wraps up our discussion of ListBoxes and ComboBoxes. Let's jump to the outside and review what we learned.

Okay, pretty important stuff … ListBoxes and ComboBoxes. The ListBox class allows for multiple selections while the ComboBox class allows for only a single item selection. So why are these classes important? Because they ensure data quality. Put a TextBox on the Form and you will get Chicago spelled a hundred different ways. Read that city's name from a database and then display it in a ComboBox and you are guaranteed consistent spelling. We will learn about relational databases in the next section but for now just know that the quality of the data in that guy will determine the ultimate success of your application. That coming from a guy who sat on both sides of the application fence, code and database. Trust me, all the pretty code is worthless if you are storing junk.

Next chapter is online learning and testing … Rick's favorite so stay tuned.